

Unit 5

Exception Handling: Exception Hierarchy, Fundamentals, Consequences of an Uncaught Exception, Handling errors, Multiple Catch, Throwing and Rethrowing an Exception, Throwable, using finally, using throws, Creating Exception Subclasses.

Exception Handling

The **Exception Handling** in Java is one of the powerful *mechanism to handle the runtime errors* so that the normal flow of the application can be maintained.

Advantage of Exception Handling

The core advantage of exception handling is to maintain the normal flow of the application. An exception normally disrupts the normal flow of the application; that is why we need to handle exceptions. Let's consider a scenario:

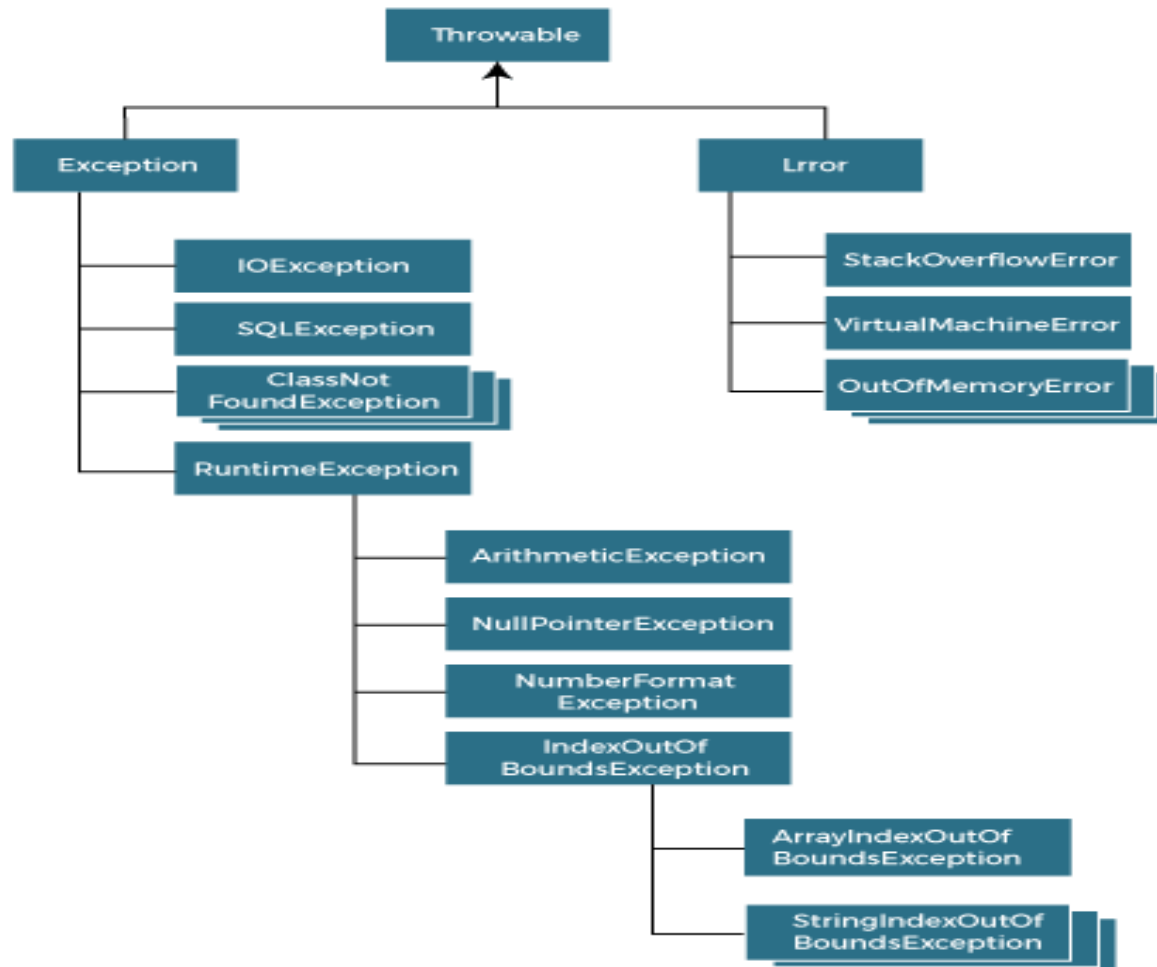
```
statement 1;  
statement 2;  
statement 3;  
statement 4;  
statement 5; //exception occurs  
statement 6;  
statement 7;  
statement 8;  
statement 9;  
statement 10;
```

Suppose there are 10 statements in a Java program and an exception occurs at statement 5; the rest of the code will not be executed, i.e., statements 6 to 10 will not be executed. However, when we perform exception handling, the rest of the statements will be executed. That is why we use exception handling in [Java](#)

Exception Handling

Hierarchy of Java Exception classes

The `java.lang.Throwable` class is the root class of Java Exception hierarchy inherited by two subclasses: `Exception` and `Error`. The hierarchy of Java Exception classes is given below:

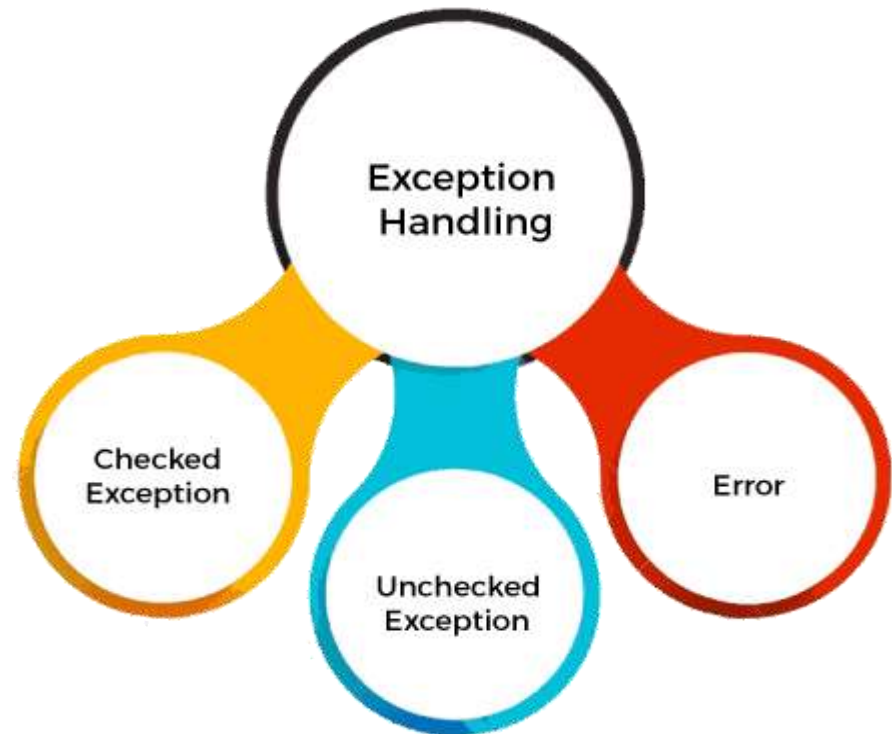


Exception Handling

Types of Java Exceptions

*There are mainly two types of exceptions: **checked** and **unchecked**. An error is considered as the unchecked exception. However, according to Oracle, there are three types of exceptions namely:*

- 1. Checked Exception*
- 2. Unchecked Exception*
- 3. Error*



Exception Handling

1) *Checked Exception*

The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, **IOException**, **SQLException**, **ClassNotFoundException** etc. Checked exceptions are checked at compile-time.

2) **Unchecked Exception**

The classes that inherit the RuntimeException are known as unchecked exceptions. For example, **ArithmeticException**, **NullPointerException**, **ArrayIndexOutOfBoundsException**, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.

3) **Error**

Error is irrecoverable. Some example of errors are **OutOfMemoryError**, **VirtualMachineError**, **AssertionError** etc

Exception Handling

Java provides five keywords that are used to handle the exception.

Keyword	Description
try	The "try" keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The try block must be followed by either catch or finally.
catch	The "catch" block is used to handle the exception. It must be preceded by try block which means we can't use catch block alone. It can be followed by finally block later.
finally	The "finally" block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The "throw" keyword is used to throw an exception.
throws	The "throws" keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It doesn't throw an exception. It is always used with method signature.

Exception Handling

```
try {  
    // block of code to monitor for errors  
}  
  
catch (ExceptionType1 exOb) {  
    // exception handler for ExceptionType1  
}  
  
catch (ExceptionType2 exOb) {  
    // exception handler for ExceptionType2  
}  
// ...  
finally {  
    // block of code to be executed after try block ends  
}
```

Exception Handling

Problem without exception handling

Let's try to understand the problem if we don't use a try-catch block

```
public class TryCatchExample1 {  
  
    public static void main(String[] args) {  
  
        int data=50/0; //may throw exception  
  
        System.out.println("rest of the code");  
  
    }  
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
```

```
public class TryCatchExample2 {  
  
    public static void main(String[] args) {  
        try  
        {  
            int data=50/0; //may throw exception  
        }  
  
        //handling the exception  
        catch(ArithmeticException e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Output:

```
java.lang.ArithmeticException: / by zero  
rest of the code
```


Exception Handling

```
public class TryCatchExample9 {  
  
    public static void main(String[] args) {  
        try  
        {  
            int arr[] = {1,3,5,7};  
            System.out.println(arr[10]); //may throw exception  
        }  
        // handling the array exception  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Exception Handling

Multiple Catch

```
public class MultipleCatchBlock1 {  
    public static void main(String[] args) {  
        try{  
            int a[]=new int[5];  
            a[5]=30/0;  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println("Arithmetic Exception occurs");  
        }  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println("ArrayIndexOutOfBoundsException occurs");  
        }  
        catch(Exception e)  
        {  
            System.out.println("Parent Exception occurs");  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Exception Handling

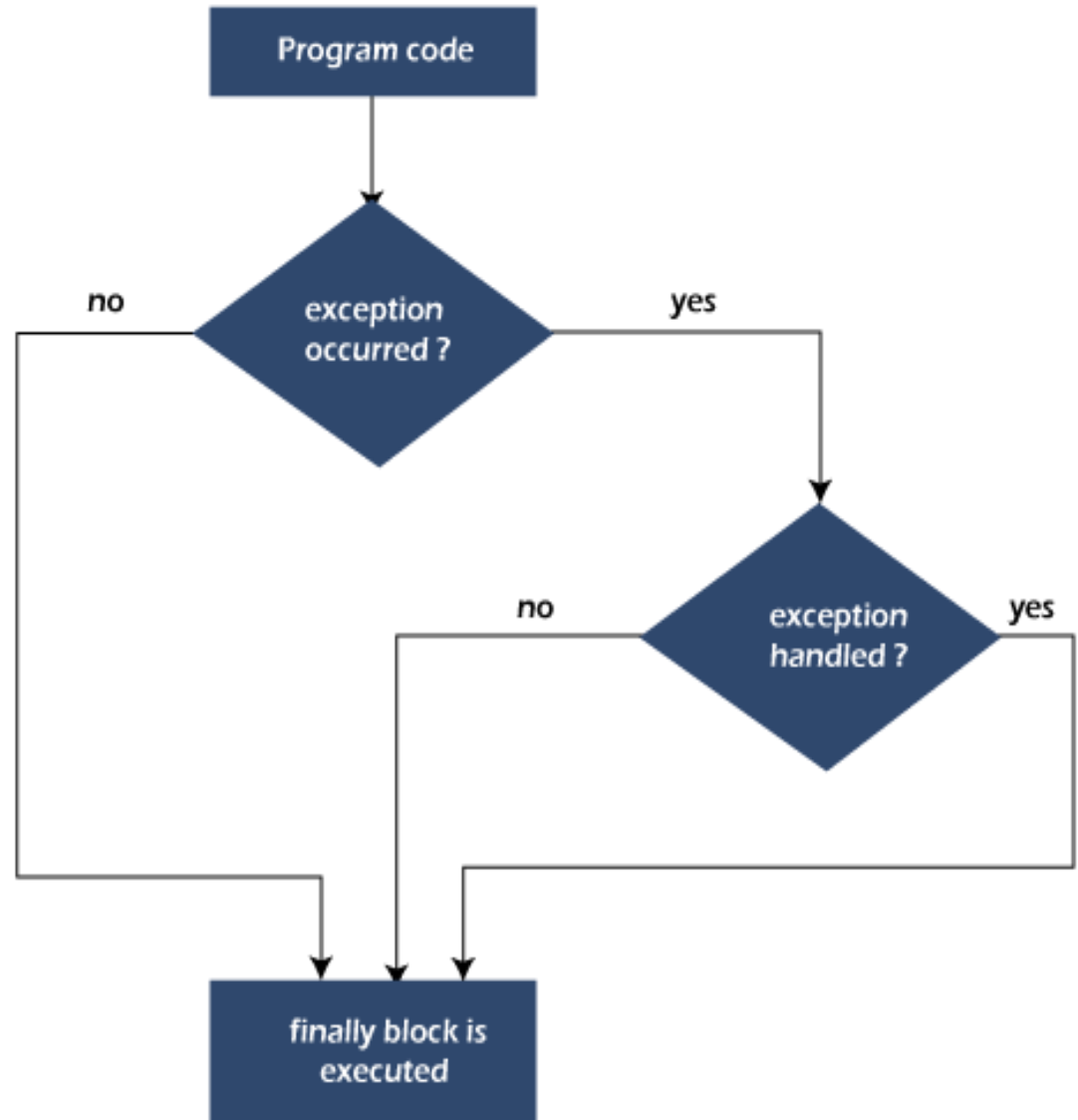
Java throw

```
public class TestThrow1 {  
    //function to check if person is eligible to vote or not  
    public static void validate(int age) {  
        if(age<18) {  
            //throw Arithmetic exception if not eligible to vote  
            throw new ArithmeticException("Person is not eligible to vote");  
        }  
        else {  
            System.out.println("Person is eligible to vote!!");  
        }  
    }  
    //main method  
    public static void main(String args[]){  
        //calling the function  
        validate(13);  
        System.out.println("rest of the code...");  
    }  
}
```

Exception Handling

finally block: java finally block is a block used to execute important code such as closing the connection, etc.

Java finally block is always executed whether an exception is handled or not. Therefore, it contains all the necessary statements that need to be printed regardless of the exception occurs or not



Exception Handling

```
class TestFinallyBlock {  
    public static void main(String args[]){  
        try{  
            //below code do not throw any exception  
            int data=25/5;  
            System.out.println(data);  
        }  
        //catch won't be executed  
        catch(NullPointerException e){  
            System.out.println(e);  
        }  
        //executed regardless of exception occurred or not  
        finally {  
            System.out.println("finally block is always executed");  
        }  
        System.out.println("rest of the code...");  
    }  
}
```

Why use Java finally block?

finally block in Java can be used to put "cleanup" code such as closing a file, closing connection, etc.

The important statements to be printed can be placed in the finally block.

Exception Handling

Sr. no.	Basis of Differences	throw	throws
1.	Definition	Java throw keyword is used throw an exception explicitly in the code, inside the function or the block of code.	Java throws keyword is used in the method signature to declare an exception which might be thrown by the function while the execution of the code.
2.	Type of exception Using throw keyword, we can only propagate unchecked exception i.e., the checked exception cannot be propagated using throw only.	Using throws keyword, we can declare both checked and unchecked exceptions. However, the throws keyword can be used to propagate checked exceptions only.	
3.	Syntax	The throw keyword is followed by an instance of Exception to be thrown.	The throws keyword is followed by class names of Exceptions to be thrown.
4.	Declaration	throw is used within the method.	throws is used with the method signature.
5.	Internal implementation	We are allowed to throw only one exception at a time i.e. we cannot throw multiple exceptions.	We can declare multiple exceptions using throws keyword that can be thrown by the method. For example, main() throws IOException, SQLException.

Exception Handling

```
public class TestThrow {  
    //defining a method  
    public static void checkNum(int num) {  
        if (num < 1) {  
            throw new ArithmeticException("\nNumber is negative, cannot calculate square");  
        }  
        else {  
            System.out.println("Square of " + num + " is " + (num*num));  
        }  
    }  
    //main method  
    public static void main(String[] args) {  
        TestThrow obj = new TestThrow();  
        obj.checkNum(-3);  
        System.out.println("Rest of the code..");  
    }  
}
```

Exception Handling

```
public class TestThrows {  
    //defining a method  
    public static int divideNum(int m, int n) throws ArithmeticException {  
        int div = m / n;  
        return div;  
    }  
    //main method  
    public static void main(String[] args) {  
        TestThrows obj = new TestThrows();  
        try {  
            System.out.println(obj.divideNum(45, 0));  
        }  
        catch (ArithmeticException e){  
            System.out.println("\nNumber cannot be divided by 0");  
        }  
  
        System.out.println("Rest of the code..");  
    }  
}
```